

CKAD



Non enregistrée, volontairement.

Pour poser vos questions, tester, vous tromper sans pression.

02:00:00

2 heures. Un terminal. Zéro hasard.

Aujourd'hui : on comprend le terrain, on installe la méthode

*“Le CKAD ne teste pas ce que vous savez.
Il teste ce que vous savez faire vite, proprement, sous contrainte.”*

```
candidate@ckad — bash

$ kubectl get pods -n exam

NAME      READY STATUS
api-7d9f  0/1   CrashLoopBackOff

$ _
```

Du backend au terminal CKAD

LB

01 Backend

Java · Spring Boot · APIs ·
microservices

02 Réalité projet

Projets clients, contraintes
réelles, prod sous pression.

03 Prépa certifs

Cloud, Kubernetes, IA : surtout
une méthode de préparation.

Ma mission : vous faire gagner en vitesse, méthode et confiance.

Votre point de départ

- 1 Votre rôle actuel
- 2 Votre niveau Kubernetes : débutant / pratique / prod
- 3 Votre objectif CKAD : certif, projet, carrière, confiance
- 4 Votre plus grosse peur pour l'examen

Le CKAD en une phrase : faire, vite, juste

100%

pratique

2h

chrono

66%

score mini

445\$

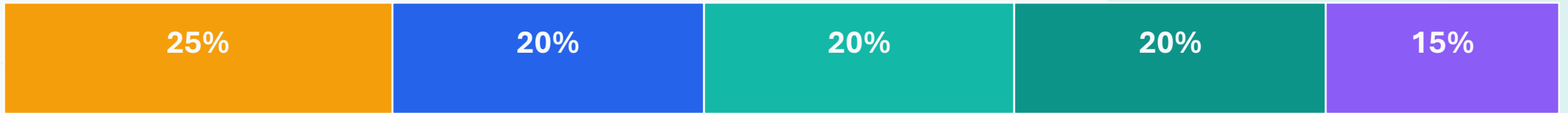
prix actuel

1

retake

Pas de QCM. Pas de hasard. Juste du terminal.

La carte du terrain : où se gagnent les points



● 25% - Environment, Configuration & Security

● 20% - Application Design & Build

● 20% - Application Deployment

● 20% - Services & Networking

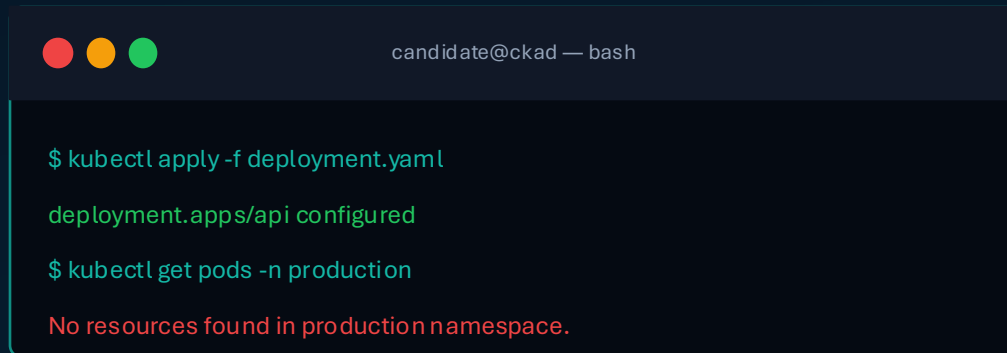
● 15% - Observability & Maintenance

“On ne révise pas tout pareil. On priorise ce qui rapporte, puis on automatise les gestes.”

Avant le terminal : une vérité sur les systèmes

Les gros incidents ne viennent pas toujours de grosses erreurs.

Ils viennent souvent d'un détail que tout le monde croyait évident :
une unité, un contexte, un namespace, un label, une variable, une indentation.



```
candidate@ckad — bash

$ kubectl apply -f deployment.yaml
deployment.apps/api configured

$ kubectl get pods -n production
No resources found in production namespace.
```

“Une commande qui réussit n’est pas encore une tâche réussie.”

: quand deux systèmes ne parlent pas la même langue

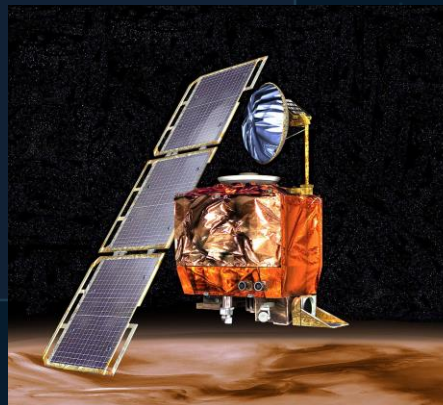
La scène

1998. Une sonde NASA arrive près de Mars pour étudier l'atmosphère.

Elle ne rate pas Mars de façon spectaculaire. Elle arrive simplement trop bas.

La cause racine rapportée : des données de propulsion en unités anglaises utilisées là où des unités métriques étaient attendues.

1 lbf·s $\approx$ 4,448 N·s



La leçon CKAD

Tout peut sembler correct localement.
Mais si le contrat entre composants est cassé, le système échoue.

En Kubernetes :

- mauvais port
- label selector qui ne match pas
- variable absente
- namespace oublié

“Le système n’a pas échoué parce qu’il ne savait pas calculer. Il a échoué parce qu’il ne parlait pas la même langue.”

Ariane 5 : le danger des hypothèses héritées



La scène

Ariane 5 a explosé lors de son premier vol en 1996 à cause d'un bug dans son logiciel de guidage.

Une partie du code venait d'Ariane 4, mais Ariane 5 volait différemment, donc certaines valeurs n'étaient plus adaptées.

Le système de navigation a planté, puis l'ordinateur a envoyé de mauvais ordres à la fusée.

Environ **37 secondes après le décollage**, elle a dévié de sa trajectoire et a été détruite en vol pour éviter un danger.
Ariane 4 : valeur horizontale < limite int16 → conversion OK

Ariane 5 : valeur horizontale > limite int16 → overflow → exception

La leçon CKAD

Le rapport pointe des erreurs de spécification et de conception dans le logiciel du système inertiel.

Leçon : une solution qui marchait dans un ancien contexte peut casser dans le nouveau.

En CKAD : ne jamais copier sans vérifier contexte, namespace, API version, labels, contraintes.

“Une hypothèse non revalidée, c'est une future panne.”

Retour Kubernetes : le détail qui casse tout

Dans notre mini-démo, le problème n'est pas spectaculaire.

1 Mars Climate Orbiter

Unités incohérentes

Le système interprète mal la donnée

2 Ariane 5

Hypothèse héritée

Le contexte a changé

3 Démo CKAD

REDIS_URL absente

Le container démarre, puis crash

“Le détail n'est pas décoratif. Dans un système, le détail fait partie du contrat.”

Mission : un pod cassé, 8 minutes pour le sauver



candidate@ckad — bash

```
$ kubectl get pods -n ckad-s1  
  
NAME      READY STATUS  
worker-7d9f8c 0/1   CrashLoopBackOff  
  
$ kubectl logs -n ckad-s1 -l app=worker --previous  
  
Error: REDIS_URL env var not set  
  
$ kubectl set env deployment/worker REDIS_URL=redis:6379 -n ckad-s1  
  
deployment.apps/worker env updated  
  
$ kubectl rollout status deploy/worker -n ckad-s1  
  
deployment successfully rolled out
```

Objectif

Trouver la cause, corriger, prouver que c'est réparé.

Méthode

Observe → Diagnose → Patch → Verify

Erreur volontaire

Corriger sans namespace pour montrer le piège classique.

La méthode CKAD : ODPV



Observe

get pods, contexte, namespace



Diagnose

describe, logs, events



Patch

corriger vite et proprement

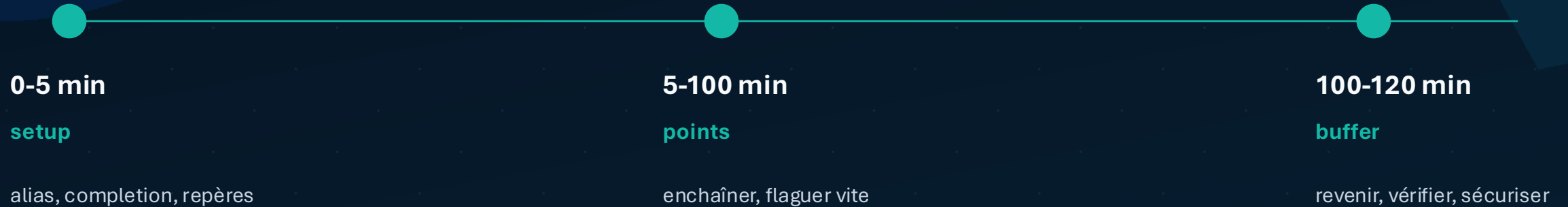


Verify

prouver que c'est OK

“Une tâche non vérifiée reste une tâche à risque.”

Le CKAD se gagne aussi en sachant passer



“Si une tâche bloque plus de 5 minutes, flag, passe, reviens. Le héros CKAD n’est pas celui qui s’acharne : c’est celui qui sécurise les points.”

Notre pacte : pas de passagers, que des pilotes

1 Questions tôt

Si vous vous la posez, quelqu'un d'autre aussi.

2 Hands-on > slides

Moins de théorie décorative, plus de gestes.

3 Erreurs bienvenues ici

On se trompe ici pour sauver des points le jour J.

4 Chrono respecté

1h courte : on va droit au résultat.

5 Livrable à chaque séance

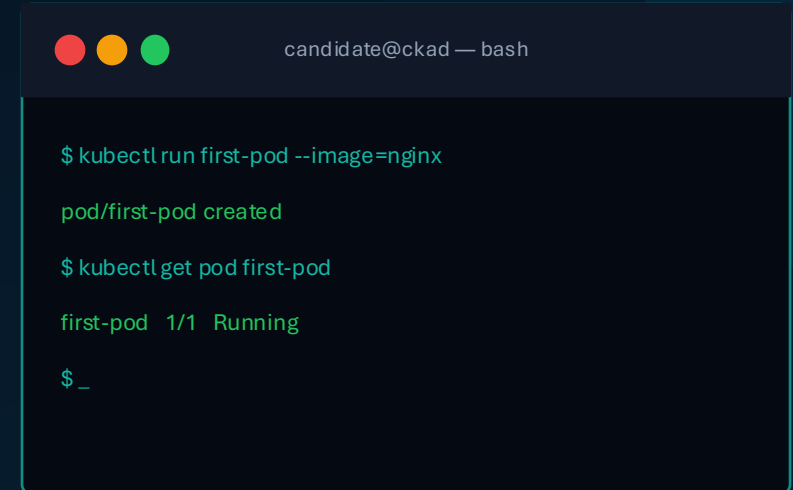
Commandes, YAML, méthode, corrigés.

6 Objectif : réussir l'examen

Pas comprendre "en général" : réussir sous contrainte.

Prochaine étape : entrer dans le terminal

- ✓ kubectl en réflexe
- ✓ YAML sans panique
- ✓ diagnostic rapide
- ✓ stratégie de temps
- ✓ vérification systématique

A terminal window with a dark background and light green text. The title bar shows 'candidate@ckad — bash'. The terminal content shows a sequence of kubectl commands and their outputs: 'kubectl run first-pod --image=nginx' followed by 'pod/first-pod created', then 'kubectl get pod first-pod' followed by 'first-pod 1/1 Running', and finally a prompt '\$ _'.

```
candidate@ckad — bash

$ kubectl run first-pod --image=nginx
pod/first-pod created

$ kubectl get pod first-pod
first-pod 1/1 Running

$ _
```

“La semaine prochaine : Pods, Containers & YAML. On démarre dans le terminal dès la minute 1.”